Integration of the HIP Portability Layer into the ACTS Parallelization Project

Summer Internship Report

European Organization For Nuclear Research (CERN)

Yasmine Badr Elhouda Larbaoui
ESI Algiers – Higher National School of Computer Science
ky_larbaoui@esi.dz

Supervised by
Joana Niermann
European Organization for Nuclear Research (CERN)

June 2025 - August 2025

Abstract

This work was carried out within the ACTS [1] development team at CERN as part of the 2025 Summer Student Program.

This report presents the integration of the HIP [2] portability layer into the ACTS parallelization project at CERN. ACTS (A Common Tracking Software) is a high-energy physics toolkit for charged-particle tracking, which includes detray [3], a modern GPU-friendly tracking detector description and propagation library.

The objective of this project was to extend GPU support beyond NVIDIA's CUDA [4] by enabling HIP, thereby supporting both NVIDIA and AMD accelerators with a unified codebase. Concretely, the work involved configuring HIP support in the build system, porting CUDA kernels to HIP while maintaining single-source files, and validating the integration through unit and integration tests within detray.

Benchmarks were executed on an AMD EPYC 7413 CPU, an NVIDIA RTX A5000 GPU, and an AMD Radeon RX 6700 XT GPU. Comparisons were made between CUDA, HIP on NVIDIA, and HIP on AMD GPUs. Results show that HIP on NVIDIA achieves performance nearly identical to CUDA, while HIP on AMD delivers competitive throughput, confirming cross-platform efficiency.

1 Introduction

Charged-particle tracking is a central task in high-energy physics (HEP): reconstructing the trajectories of charged particles through a detector is essential for event reconstruction and physics analysis. This process requires propagating millions of tracks through complex geometries and magnetic fields, making computational efficiency crucial.

Graphics Processing Units (GPUs) have become indispensable for high-performance computing due to their massive parallelism and high throughput.

In HEP experiments, track reconstruction requires propagating millions of particle trajectories through complex detector geometries. GPUs, with their parallel architecture, provide a way to accelerate track reconstruction workloads , they can execute these independent propagations simultaneously, drastically reducing processing time.

However, the GPU programming ecosystem has long been dominated by NVIDIA's CUDA, which delivers excellent performance but locks applications to NVIDIA hardware. This vendor dependence is problematic in modern heterogeneous computing environments, where both NVIDIA and AMD accelerators are deployed in supercomputing facilities.

To address this limitation, AMD developed HIP, a CUDA-like programming model and runtime API that allows code to be compiled and executed on both NVIDIA GPUs and AMD GPUs .

This project focuses on extending GPU support in detray . My contribution was focused on implementing the HIP backend within detray, adapting existing CUDA kernels, and validating their execution on both AMD and NVIDIA architectures. This integration aimed to ensure full portability without compromising numerical accuracy.

2 Technical Background

2.1 GPU Programming Models

Modern GPU programming models are typically vendor-specific. NVIDIA's *Compute Unified Device Architecture* (CUDA) is a proprietary framework optimized exclusively for NVIDIA GPUs, providing extensive libraries and tooling support for high-performance computing.

In contrast, AMD developed *HIP* (Heterogeneous-computing Interface for Portability), a C++ runtime API and kernel language that mirrors the CUDA programming model. HIP allows developers to target both:

- **NVIDIA GPUs**, by compiling HIP code through the CUDA backend, and

- **AMD GPUs**, via the ROCm [5] (Radeon Open Compute) toolchain.

HIP acts as a lightweight abstraction layer as presented in Figure 1 , that translates kernel calls to the appropriate backend while preserving the same parallel execution model and memory hierarchy. This enables developers to maintain nearly identical source code for heterogeneous GPU environments without major code duplication.

2.2 Portability in Scientific Software

For large-scale scientific frameworks such as ACTS, maintaining performance portability ,

achieving efficient execution on multiple architectures without rewriting the code is essential for long-term sustainability.

To address this need, HIP was selected within the CERN software stack because it:

- Supports both NVIDIA and AMD hardware architectures,

- Integrates seamlessly with both ROCm and CUDA toolchains.

ROCm (Radeon Open Compute) is AMD's open software platform that provides compiler, runtime, and debugging tools for GPU computing.

- Enables incremental migration from existing CUDA codebases using tools such as `hipify`, which automatically translates CUDA syntax into HIP.

This approach ensures hardware flexibility, reduces vendor lock-in, and provides a practical pathway toward cross-platform GPU acceleration in high-energy physics applications.

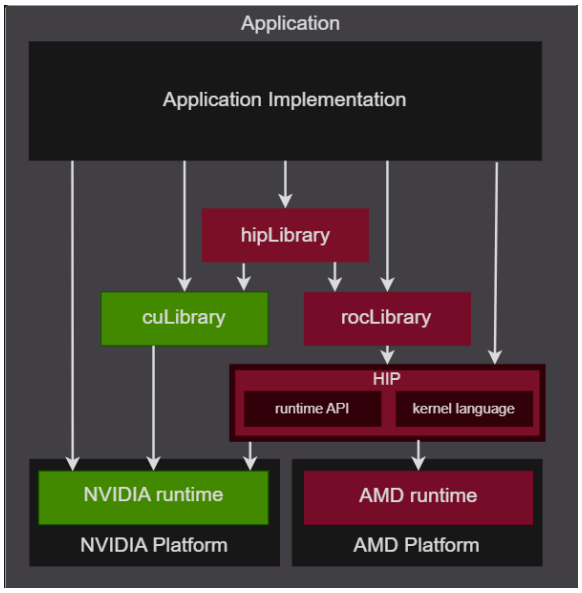

Figure 1: HIP Portability Layer for Heterogeneous GPU Platforms.

In practice, these portability mechanisms directly influenced the integration strategy in this project. Understanding HIP's dual backend behavior (ROCm vs CUDA) was crucial when adapting the build system and ensuring consistent compiler flags across platforms.

3 GPU Acceleration in ACTS/Detray

3.1 ACTS and Detray

ACTS (A Common Tracking Software) is a modular C++ toolkit for charged-particle tracking in HEP experiments. Within ACTS :

- **Detray**: a geometry and tracking toolkit designed to run efficiently on modern hardware, providing:
  - Propagation of charged particles in magnetic fields.
  - Covariance transport for error estimation,
  - Material interaction modeling during propagation.

  Written in C++20 with a flat memory layout, detray is designed for parallel execution and GPU acceleration, making it suitable for workloads involving millions of track propagations.
- **Vecmem** [6]: a memory management library that abstracts host and device memory resources (e.g., `vecmem::hip` for HIP backends). It provides the foundation for allocating, transferring, and synchronizing data between CPU and GPU.
- **Propagation Kernels**: GPU kernels implementing physics algorithms such as track propagation, Runge-Kutta integration steps, and handling of detector geometry interactions.

In this internship, I specifically worked on detray's GPU acceleration layer , modifying its build logic and kernel implementations to support HIP while maintaining compatibility with existing CUDA and SYCL backends.

4 Methodology

4.1 HIP Integration into the Build System

The integration of HIP into the `detray` build system began by modifying the CMake [7] configuration to define HIP as a new accelerator backend. CMake is a cross-platform build automation tool that generates native build files (e.g., Makefiles or Ninja files). It is widely used in high-energy physics projects to ensure consistent compilation across different operating systems, compilers, and hardware backends.

In this project, the CMake configuration was updated to integrate HIP alongside CUDA and SYCL, while ensuring full compatibility with modern C++ standards and GPU compilers. This change was necessary because Detray's build system originally recognized only CUDA and SYCL. Therefore, supporting HIP required defining new compiler targets, build flags, and detection logic within the CMake scripts.

The modifications were applied at two levels: the main `CMakeLists.txt` file and a dedicated module `hip-compiler-options.cmake`.

HIP was introduced as a first-class language with the following configurations:

- **C++20 Standard Enforcement:** The `detray` codebase relies on C++20 features (e.g., concepts and ranges).Figure 2 illustrates the enforcement of the C++20 standard in the HIP compiler configuration , which ensured that HIP device code behaved consistently with the rest of the detray codebase. It also prevents builds with outdated compiler versions or missing C++20 support. These

safeguards protect users from attempting to compile in environments that are fundamentally incompatible (e.g., older ROCm toolchains).

```
if(${DETRAY_BUILD_HIP} AND ${CMAKE_HIP_STANDARD} LESS 20)
    message(
        SEND_ERROR
        "CMAKE_HIP_STANDARD=${CMAKE_HIP_STANDARD}, but detray requires C++>=20"
    )
endif()
```

Figure 2: C++20 Enforcment for HIP in Detray.

- **Architecture Targeting:** Figure 3 presents the architecture targeting for HIP compilation on AMD GPUs. This step ensures that the generated code is optimized for the specific hardware instruction set.

```
set(CMAKE_HIP_ARCHITECTURES gfx1031)

set(CMAKE_HIP_STANDARD 20 CACHE STRING "The (HIP) C++ standard to use")
set(CMAKE_HIP_EXTENSIONS FALSE CACHE BOOL "Disable (HIP) C++ extensions")
```

Figure 3: CMake configuration for HIP Architecture and C++ Standard settings.

- **Build Control Option:** A user-selectable CMake option enables or disables HIP support (Figure 4) , explicitly control over whether HIP code is compiled or not. This is important because not all users of `detray` have HIP or ROCm installed on their systems. The build option prevents unnecessary compilation errors and allows users to enable HIP only if their environment supports it.

```
option(DETRAY_BUILD_HIP "Build the HIP sources included in detray" OFF)
```

Figure 4: CMake option to enable or disable HIP support.

- **Conditional Build Integration:** Figure 5 depicts the conditional integration of HIP with other backends : HIP was added alongside CUDA and SYCL in the list of accelerator backends , for instance, the propagation benchmarks and unit tests and integration tests in `detray` can now be compiled either with CUDA, HIP, or SYCL, depending on user configuration.

```
cmake_dependent_option(
    DETRAY_BUILD_HOST
    "Build the host sources included in detray"
    ON
    "DETRAY_BUILD_CUDA OR DETRAY_BUILD_SYCL OR DETRAY_BUILD_HIP"
    ON
)
```

Figure 5: Conditional integration of HIP with other accelerator backends.

- **Language Detection (vecmem):** HIP support required integration with `vecmem`, a C++ memory management library designed for heterogeneous programming in HEP applications . In `detray`, `vecmem` provides device memory resources for HIP, CUDA, and host code. Language detection in CMake ensures that HIP can be used consistently with `vecmem`'s abstractions for buffer allocation and memory transfer (Language detection in vecmem is illustrated in Figure 6.).

```
# Check if HIP is available
include(vecmem-check-language)
```

Figure 6: checking languages in vecmem.

- **Centralized Compiler Options:** The `hip-compiler-options.cmake` module was created to centralize HIP compiler flags for both AMD ROCm and HIP-enabled NVIDIA compilers. By ensuring consistency across platforms, this avoids divergent behavior in error handling, constexpr function calls, or debug symbol generation. For example, the `-expt-relaxed-constexpr` flag necessary in CUDA when calling certain `constexpr` host functions inside kernels was also applied in HIP builds to maintain functional equivalence.

  Figures 7 and 8 summarize the compiler configurations for AMD and NVIDIA platforms, respectively.

```
if(
    ("${CMAKE_HIP_PLATFORM}" STREQUAL "hcc")
    OR ("${CMAKE_HIP_PLATFORM}" STREQUAL "amd")
)
    detray_add_flag( CMAKE_HIP_FLAGS "-Wall" )
    detray_add_flag( CMAKE_HIP_FLAGS "-Wextra" )
    detray_add_flag( CMAKE_HIP_FLAGS "-Wshadow" )
    detray_add_flag( CMAKE_HIP_FLAGS "-Wunused-local-typedefs" )
    detray_add_flag( CMAKE_HIP_FLAGS "-pedantic" )
endif()
```

Figure 7: HIP compiler configuration for AMD platform.

```
if(
    ("${CMAKE_HIP_PLATFORM}" STREQUAL "nvcc")
    OR ("${CMAKE_HIP_PLATFORM}" STREQUAL "nvidia")
)
    detray_add_flag( CMAKE_HIP_FLAGS_DEBUG "-G" )
    detray_add_flag( CMAKE_HIP_FLAGS "--expt-relaxed-constexpr" )
endif()
```

Figure 8: HIP compiler configuration for NVIDIA platform.

In summary, this configuration ensured that HIP builds behave consistently across AMD ROCm and

NVIDIA backends, while also aligning with the C++20 based design of `detray`.

4.2 Porting CUDA Kernels to HIP

To enable GPU acceleration beyond CUDA only environments, selected CUDA source files in `detray` were ported to HIP .

the HIP port included the detector and propagator kernels within detray::propagation, which I adapted and tested using GoogleTest-based unit tests. I validated correctness by comparing HIP outputs to CPU and CUDA references under identical geometry and field configurations.

The HIP integration into detray followed a structured workflow, beginning with unit tests, followed by integration tests, and concluding with benchmarking :

- **Integration Tests**: These tests ensure that propagator components in DetRay work as intended when integrated with other system parts. They are designed to verify that the system works cohesively.

- **Unit Tests**: These tests target the detector components in DetRay, ensuring that each isolated functionality is working properly.

1. Build System Integration:

- Configure CMake to enable HIP support, including compiler options and target GPU architectures.

- Define unit and integration tests for continuous verification of correctness and

- Link required libraries such as :

  – `vecmem::hip`: a memory management library used by `detray` to handle GPU/CPU memory allocation and transfers,

  – GTest: the GoogleTest framework, used to implement unit and integration tests for correctness of geometry and propagation.

An example of this setup and configuration steps are shown in Figures 9 and 10 for the unit tests part.

```
message(STATUS "Building detray HIP unit tests")
cmake_minimum_required(VERSION 3.21) # HIP langauge support requires minimum 3.21

find_package(HIPToolkit)

# Enable HIP as a language.
enable_language(HIP)

include(detray-compiler-options-hip)
```

Figure 9: Enabling HIP Language Support in the CMake Configuration.

```
detray_add_unit_test(hip_${CMAKE_HIP_PLATFORM}_${algebra}
    "detector_hip_kernel.hpp"
    "detector_hip.cpp"
    "detector_hip_kernel.hip"

    LINK_LIBRARIES GTest::gtest_main vecmem::hip  detray::core
    detray::algebra_${algebra} detray::test_common detray::test_utils HIP::hiprt
)

# Make hipcc interpret .cpp files as .hip files to make linking work
# (only needed for the amd backend)
if(
    ("${CMAKE_HIP_PLATFORM}" STREQUAL "hcc")
    OR ("${CMAKE_HIP_PLATFORM}" STREQUAL "amd")
)
    set_source_files_properties(detector_hip.cpp PROPERTIES LANGUAGE HIP)
endif()
```

Figure 10: Defining Detector HIP Unit Tests for Multiple Algebras .

2. Memory Management:

- Allocate device and managed memory using **`vecmem::hip::device_memory_resource`** and **`vecmem::hip::managed_memory_resource`**

In the Vecmem library, a memory resource is an abstraction for memory allocation, similar to the C++17 std::pmr::memory_resource. The class vecmem::hip::managed_memory_resource specifically provides allocations backed by HIP unified (managed) memory. Internally, it calls hipMallocManaged, which creates memory that is accessible by both the host (CPU) and the device (GPU) without the need for explicit copy operations. This is particularly useful in the Detray HIP port, since geometry objects and detector data can be created on the host and then directly accessed or modified on the GPU.

By contrast, vecmem::hip::device_memory_resource allocates GPU-only memory (hipMalloc), which requires explicit host-to-device transfers. Using managed_memory_resource allows us to simplify data movement and debugging during development, while still integrating cleanly with Vecmem containers such as vecmem::vector.

- Explicitly transfer data between host and device to respect GPU/CPU memory differences (GPU memory is non-coherent and requires explicit management).

- Synchronize kernel launches using **`hipDeviceSynchronize`** and perform error checking after each launch.

3. Kernel Porting: The porting process focused on adapting Detray's CUDA kernels to HIP. At this stage, the CUDA and HIP implementations coexist in separate files rather than following a true single-source model, but the HIP code now reproduces the same logic previously written in CUDA. Two categories of kernels were ported. First, the detector

kernels, which handle the transfer of detector geometry data (volumes, surfaces, coordinate transforms, and masks such as rectangles, discs, and cylinders) from host to device memory. These kernels rely on `vecmem hip managed_memory_resource`, an abstraction layer that manages CPU–GPU memory consistently and avoids manual memory handling. The correctness of these transfers is verified by unit tests that compare device-side geometry with the original host-side representation. Second, the propagator kernels, which advance track states through the detector geometry under magnetic fields. A track state in Detray encapsulates a particle's position, momentum, and charge, and is updated iteratively during propagation. The numerical integration is based on a Runge–Kutta algorithm, which solves the differential equations of motion in inhomogeneous fields. To ensure correctness, integration tests compare HIP-based propagation against CPU baselines, confirming numerical consistency within tolerance. Together, these results validate that the HIP port faithfully reproduces the CUDA implementation while enabling GPU execution across different hardware backends.

4. **Kernel Execution and Validation:**

- Launch detector and propagator kernels using **`hipLaunchKernelGGL`** with thread and block configurations tuned to balance occupancy and memory access efficiency on the target hardware.

  Figures 11 and 12 show kernel launch and execution traces for detector kernels.

```
// run the test kernel
hipLaunchKernelGGL(detector_test_kernel , dim3(block_dim), dim3(thread_dim) , 0, 0 ,
    det_data, volumes_data, surfaces_data, transforms_data, rectangles_data,
    discs_data, cylinders_data);
// HIP error check
DETRAY_HIP_ERROR_CHECK(hipGetLastError());

DETRAY_HIP_ERROR_CHECK(hipDeviceSynchronize());
```

Figure 11: Launching HIP kernels with appropriate grid, block, and memory configurations.

```
[ylarbaou@pcadp04 real_detray]$ ./detray_build/bin/detray_unit_test_hip_and_array
Running main() from /mnt/ssd1/ylarbaou/real_detray/detray_build/_deps/googletest-src/googletest/src/gtest_main.cc
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from detector_hip
[ RUN      ] detector_hip.detector
INFO: Checking detector consistency...
WARNING: No entries in volume finder
INFO: Consistency check: OK
[       OK ] detector_hip.detector (617 ms)
[----------] 1 test from detector_hip (617 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (617 ms total)
[  PASSED  ] 1 test.
```

Figure 12: Example of HIP kernel execution trace for detector kernel.

5 Benchmarks

To evaluate HIP performance, dedicated benchmark executables were added to the `detray` build system.

The benchmark infrastructure for the Detray propagation tests already existed in CUDA. As part of this internship, I ported the CUDA benchmark implementation to HIP, enabling cross-platform benchmarking on both AMD and NVIDIA GPUs. The port included adapting kernel launches, memory management (`hipMalloc`, `hipMemcpy`, `hipDeviceSynchronize`), and error handling macros (`DETRAY_HIP_ERROR_CHECK`). The HIP benchmark code was integrated into the Detray build system , using the same configuration and benchmarking parameters as the original CUDA version. The benchmarks were run using the Google Benchmark framework already present in the project, to measure throughput (tracks propagated per second) for different detector configurations .

5.1 HIP Propagation Benchmark Kernel

A dedicated propagation benchmark kernel was implemented in HIP (`propagation_benchmark.hip`). The kernel evaluates track propagation within `detray` geometries by instantiating propagators on the GPU and comparing synchronous vs. asynchronous propagation:

Kernel launch: `hipLaunchKernelGGL` with optimized block size (256 threads).

Detector setup: Detector views, magnetic field views, and actor states are copied to HIP-managed memory.

Propagation logic:

1. Initialize propagator state per track, setting up the particle's initial parameters (position, direction, momentum, charge).

2. Provide detector views, magnetic field views, and actor states as kernel arguments, which are passed directly to the device (typically using vecmem copy objects rather than managed memory).

3. Run the propagation kernel, which advances the particle state through the geometry and magnetic field using the configured actors. The benchmark focuses on measuring the time required to propagate a given number of particles and reports this as a propagation rate.

4. Report benchmark results, which consist of timing measurements only. No validation of physics accuracy is performed in this benchmark, since additional checks would interfere with the latency measurements.

Error handling: the `DETRAY_HIP_ERROR_CHECK` macro ensures correctness at runtime.

5.2 Benchmark Workflow

The benchmark workflow follows three stages:

- Setup :
  - Allocate hip-memory memory : `vecmem::hip::device_memory_resource` `managed_memory_resource`.
  - Copy actor states from host to device (`hipMemcpy`).
  - Configure geometry, material, and grid files.
- Execution :
  - Launch benchmark kernel with a configurable number of tracks (`n_samples`).
  - Track propagation under magnetic fields is executed on GPU.
- Validation :
  - Compare HIP outputs against CPU reference results.
  - Measure throughput and runtime using `Google Benchmark`.
  - Dump results in JSON format for plotting and analysis (`-benchmark_out`).

5.3 Benchmark Results

Benchmarks were executed on three different hardware setups:

- **CPU**: AMD EPYC 7413 .
- **NVIDIA GPU**: RTX A5000 (CUDA & HIP backends).
- **AMD GPU**: Radeon RX 6700 XT (HIP backend).

The benchmark runs generate detailed JSON output files containing the measured propagation throughput for a given detector geometry, magnetic field, and physics configuration. The JSON files are subsequently used to create plots that visualize performance across different backends and algebra plugins.

Option	Description
–geometry_file	JSON description of the detector geometry.
–grid_file	Precomputed navigation grids for spatial acceleration.
–material_file	Material description (used for multiple scattering, energy loss, covariance transport).
–sort_tracks	Orders tracks for improved memory access and reproducibility.
–randomize_charge	Randomizes track charge to avoid systematic bias.
–eta_range -3 3	Pseudorapidity range of generated test tracks.
–pT_range 0.5 100	Transverse momentum range [GeV].
–search_window 3 3	Lookup window size for navigation in acceleration grids.
–covariance_transport	Enables propagation of track covariance matrices with material effects.
–bknd_name <backend>	Label identifying the benchmarked backend (e.g. cuda_array, hip_amd_array).
–benchmark_repetitions=5	Number of repeated runs for averaging.
–benchmark_out	Output file in JSON for later analysis.

Table 1: Benchmark configuration options and their descriptions

5.4 Benchmark Metrics :

To evaluate the performance of the propagation backends, we use the *throughput* as the main metric. Throughput measures how many tracks can be propagated per unit time, and is defined as

$$\text{Throughput} = \frac{N_{\text{tracks}}}{t_{\text{exec}}}, \tag{1}$$

where N_{tracks} is the number of propagated tracks and t_{exec} s is the execution time. In the benchmark plots, throughput is expressed in millions of tracks per second (MHz). The x-axis shows the number of tracks being propagated, while the y-axis shows the resulting throughput. This allows us to study how different backends scale with increasing workload sizes. For compar-

ison, latency (the time required to propagate a single track) is the inverse of throughput.

The following plots (Figures 13-18) show throughput and latency comparisons between HIP, CUDA, and CPU backends using single precision on both NVIDIA and AMD hardware.

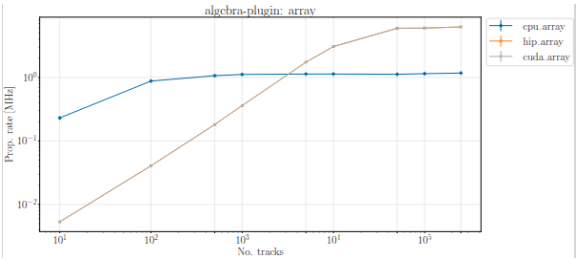

Figure 13: Throughput Comparison of HIP & CUDA & CPU performance on NVIDIA Backends

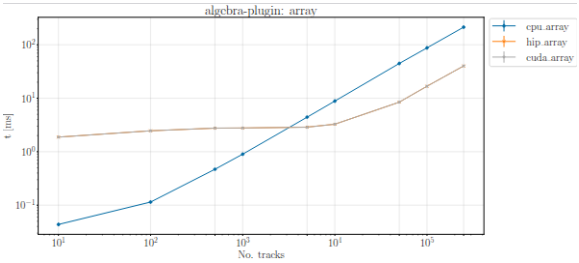

Figure 14: Latency Comparison of HIP & CUDA & CPU performance on NVIDIA Backends

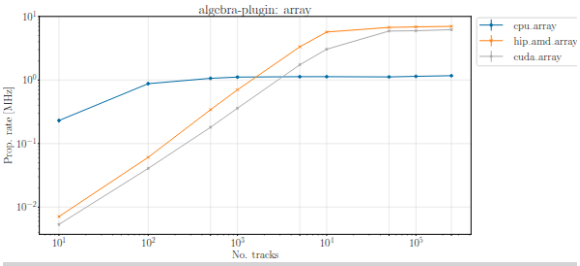

Figure 15: Throughput Comparison of HIP (AMD) & CUDA (NVIDIA) & CPU

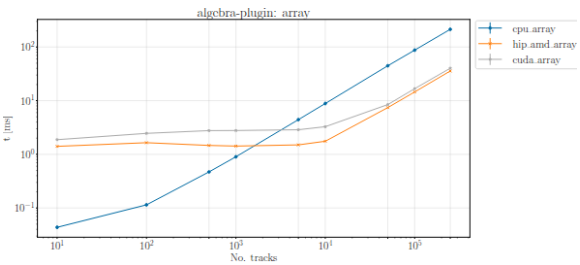

Figure 16: Latency Comparison of HIP (AMD) & CUDA (NVIDIA) & CPU

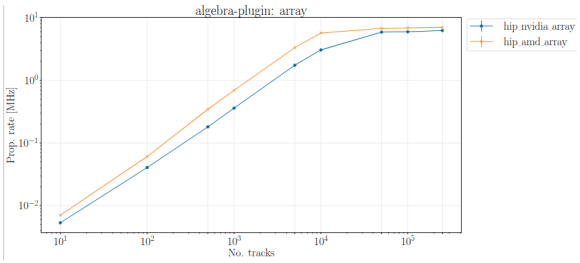

Figure 17: Throughput Comparison of HIP (AMD) & HIP (NVIDIA)

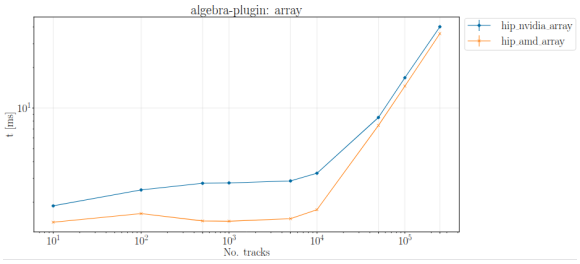

Figure 18: Latency Comparison of HIP (AMD) & HIP (NVIDIA)

To further assess numerical performance, an additional benchmark was conducted on both NVIDIA and AMD backends comparing single vs. double precision (see Figure 19).

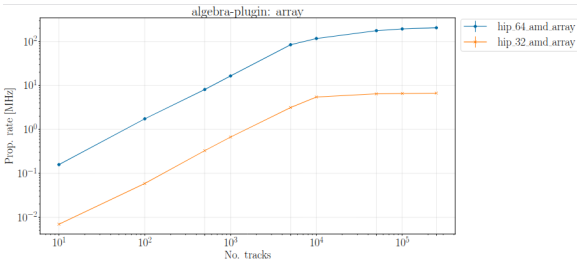

Figure 19: Throughput Comparison of HIP on AMD : Single precision vs Double precision

6 Results and Discussion

The benchmark results demonstrate consistent and portable performance across all tested backends:

- For small numbers of tracks, the CPU executes faster due to the GPU initialization overhead.

- As the workload increases (thousands of tracks or more), GPUs become significantly faster.

- HIP on NVIDIA performs comparably to native CUDA once beyond the small-track regime, confirming the correctness and efficiency of the portability layer.

- CUDA and HIP on NVIDIA exhibit nearly identical performance, demonstrating that HIP introduces no measurable overhead.

Regarding HIP on AMD versus NVIDIA:

- For medium workloads, AMD performs marginally better.

- For large workloads, both GPUs reach similar peak throughput.

Unexpected behavior was observed in double precision benchmarks, where performance differed more significantly. On the AMD GPU, double precision propagation consistently outperformed single precision.

This anomaly opens the door to new research avenues: analyzing why AMD GPUs favor double precision in this context .

Overall, this work validates HIP as a viable abstraction for cross-platform GPU programming in ACTS, while also revealing unexpected precision-related phenomena that could inspire future research.

7 Conclusion and Future Work

This project successfully integrated HIP into the detray library, enabling ACTS to run on both NVIDIA and AMD GPUs. Key outcomes include:

- HIP support in the build system through CMake.
- Porting of CUDA kernels to HIP .
- Unit and integration tests validating correctness .
- Benchmarks demonstrating performance portability.

Future work should focus on:

- Optimizing HIP kernels specifically for AMD hardware,
- Extending HIP integration to other components of ACTS,
- Comparing HIP with alternative portability solutions such as SYCL,
- Investigating performance behavior in double precision,
- Exploring performance tuning for specific geometries and magnetic field configurations.

Through this project, I gained hands-on experience in heterogeneous computing, build system configuration, and performance benchmarking across multiple GPU architectures. The HIP integration I implemented is now part of detray's continuous integration pipeline, supporting future portability studies within ACTS.

8 Acknowledgements

I would like to express my sincere gratitude to my supervisor Joana Niermann at CERN for her exceptional guidance and support throughout this project. I also extend my thanks to the entire ACTS development team for their technical assistance and valuable insights. This internship has been an invaluable learning experience, both professionally and personally.

References

[1] ACTS Collaboration. (2023). *A Common Tracking Software (ACTS).* `https://github.com/acts-project/acts` `https://link.springer.com/article/10.1007/s41781-021-00078-8`

[2] AMD. (2023). *HIP: Heterogeneous-Compute Interface for Portability.* `https://github.com/ROCm/HIP`

[3] Detray Development Team. (2023). *Detray: A GPU-friendly tracking detector description and propagation library.* `https://github.com/acts-project/detray` `https://iopscience.iop.org/article/10.1088/1742-6596/2438/1/012026`

[4] NVIDIA. (2023). *CUDA Toolkit Documentation.* `https://docs.nvidia.com/cuda/`

[5] AMD. (2023). *ROCm Open Software Platform.* `https://github.com/ROCm`

[6] Barisits, M. et al. (2023). *Vecmem: Memory management for HEP software.* J. Phys.: Conf. Ser., 2438, 012050.

[7] Kitware Inc. (2024). *CMake: Cross-Platform Build System.* `https://cmake.org`